

CONVEX POTENTIAL FLOWS: UNIVERSAL PROBABILITY DISTRIBUTIONS WITH OPTIMAL TRANSPORT AND CONVEX OPTIMIZATION

Chin-Wei Huang

University of Montreal & Mila
chin-wei.huang@umontreal.ca

Ricky T. Q. Chen

University of Toronto & Vector Institute
rtqichen@cs.toronto.edu

Christos Tsirigotis

University of Montreal & Mila
christos.tsirigotis@umontreal.ca

Aaron Courville

University of Montreal, Mila & CIFAR Fellow
aaron.courville@umontreal.ca

ABSTRACT

Flow-based models are powerful tools for designing probabilistic models with tractable density. This paper introduces Convex Potential Flows (CP-Flow), a natural and efficient parameterization of invertible models inspired by the optimal transport (OT) theory. CP-Flows are the gradient map of a strongly convex neural potential function. The convexity implies invertibility and allows us to resort to convex optimization to solve the convex conjugate for efficient inversion. To enable maximum likelihood training, we derive a new gradient estimator of the log-determinant of the Jacobian, which involves solving an inverse-Hessian vector product using the conjugate gradient method. The gradient estimator has *constant-memory* cost, and can be made effectively *unbiased* by reducing the error tolerance level of the convex optimization routine. Theoretically, we prove that CP-Flows are *universal* density approximators and are *optimal* in the OT sense. Our empirical results show that CP-Flow performs competitively on standard benchmarks of density estimation and variational inference.

1 INTRODUCTION

Normalizing flows (Dinh et al., 2014; Rezende & Mohamed, 2015) have recently gathered much interest within the machine learning community, ever since its recent breakthrough in modelling high dimensional image data (Dinh et al., 2017; Kingma & Dhariwal, 2018). They are characterized by an invertible mapping that can reshape the distribution of its input data into a simpler or more complex one. To enable efficient training, numerous tricks have been proposed to impose structural constraints on its parameterization, such that the density of the model can be tractably computed.

We ask the following question: “what is the natural way to parameterize a normalizing flow?” To gain a bit more intuition, we start from the one-dimension case. If a function $f : \mathbb{R} \rightarrow \mathbb{R}$ is continuous, it is invertible (injective onto its image) if and only if it is strictly monotonic. This means that if we are only allowed to move the probability mass continuously without flipping the order of the particles, then we can only rearrange them by changing the distance in between.

In this work, we seek to generalize the above intuition of monotone rearrangement in 1D. We do so by motivating the parameterization of normalizing flows from an optimal transport perspective, which allows us to define some notion of rearrangement cost (Villani, 2008). It turns out, if we want the output of a flow to follow some desired distribution, under mild regularity conditions, we can characterize the unique optimal mapping by a convex potential (Brenier, 1991). In light of this, we propose to parameterize normalizing flows by the gradient map of a (strongly) convex potential. Owing to this theoretical insight, the proposed method is provably *universal* and *optimal*; this means the proposed flow family can approximate arbitrary distributions and requires the least amount of transport cost. Furthermore, the parameterization with convex potentials allows us to formulate model inversion and gradient estimation as convex optimization problems. As such, we

make use of existing tools from the convex optimization literature to cheaply and efficiently estimate all quantities of interest.

In terms of the benefits of parameterizing a flow as a gradient field, the convex potential is an $\mathbb{R}^d \rightarrow \mathbb{R}$ function, which is different from most existing discrete-time flows which are $\mathbb{R}^d \rightarrow \mathbb{R}^d$. This makes CP-Flow relatively compact. It is also arguably easier to design a convex architecture, as we do not need to satisfy constraints such as orthogonality or Lipschitzness; the latter two usually require a direct or an iterative reparameterization of the parameters. Finally, it is possible to incorporate additional structure such as equivariance (Cohen & Welling, 2016; Zaheer et al., 2017) into the flow’s parameterization, making CP-Flow a more flexible general purpose density model.

2 BACKGROUND: NORMALIZING FLOWS AND OPTIMAL TRANSPORT

Normalizing flows are characterized by a differentiable, invertible neural network f such that the probability density of the network’s output can be computed conveniently using the change-of-variable formula

$$p_Y(f(x)) = p_X(x) \left| \frac{\partial f(x)}{\partial x} \right|^{-1} \iff p_Y(y) = p_X(f^{-1}(y)) \left| \frac{\partial f^{-1}(y)}{\partial y} \right| \quad (1)$$

where the Jacobian determinant term captures the local expansion or contraction of the density near x (resp. y) induced by the mapping f (resp. f^{-1}), and p_X is the density of a random variable X . The invertibility requirement has led to the design of many special neural network parameterizations such as triangular maps, ordinary differential equations, orthogonality or Lipschitz constraints.

Universal Flows For a general learning framework to be meaningful, a model needs to be flexible enough to capture variations in the data distribution. In the context of density modeling, this corresponds to the model’s capability to represent arbitrary probability distributions of interest. Even though there exists a long history of literature on universal approximation capability of deep neural networks (Cybenko, 1989; Lu et al., 2017; Lin & Jegelka, 2018), invertible neural networks generally have limited expressivity and cannot approximate arbitrary functions. However, for the purpose of approximating a probability distribution, it suffices to show that the distribution induced by a normalizing flow is universal.

Among many ways to establish distributional universality of flow based methods (e.g. Huang et al. 2018; 2020b; Teshima et al. 2020; Kong & Chaudhuri 2020), one particular approach is to approximate a *deterministic coupling* between probability measures. Given a pair of probability densities p_X and p_Y , a deterministic coupling is a mapping g such that $g(X) \sim p_Y$ if $X \sim p_X$. We seek to find a coupling that is invertible, or at least can be approximated by invertible mappings.

Optimal Transport Let $c(x, y)$ be a cost function. The *Monge problem* (Villani, 2008) pertains to finding the optimal transport map g that realizes the minimal expected cost

$$J_c(p_X, p_Y) = \inf_{\tilde{g}: \tilde{g}(X) \sim p_Y} \mathbb{E}_{X \sim p_X} [c(X, \tilde{g}(X))] \quad (2)$$

When the second moments of X and Y are both finite, and X is regular enough (e.g. having a density), then the special case of $c(x, y) = \|x - y\|^2$ has an interesting solution, a celebrated theorem due to Brenier (1987; 1991):

Theorem 1 (Brenier’s Theorem, Theorem 1.22 of Santambrogio (2015)). *Let μ, ν be probability measures with a finite second moment, and assume μ has a Lebesgue density p_X . Then there exists a convex potential G such that the gradient map $g := \nabla G$ (defined up to a null set) uniquely solves the Monge problem in eq. (2) with the quadratic cost function $c(x, y) = \|x - y\|^2$.*

Some recent works are also inspired by Brenier’s theorem and utilize a convex potential to parameterize a critic model, starting from Taghvaei & Jalali (2019), and further built upon by Makkuva et al. (2019) who parameterize a generator with a convex potential and concurrently by Korotin et al. (2019). Our work sets itself apart from these prior works in that it is entirely likelihood-based, minimizing the (empirical) KL divergence as opposed to an approximate optimal transport cost.

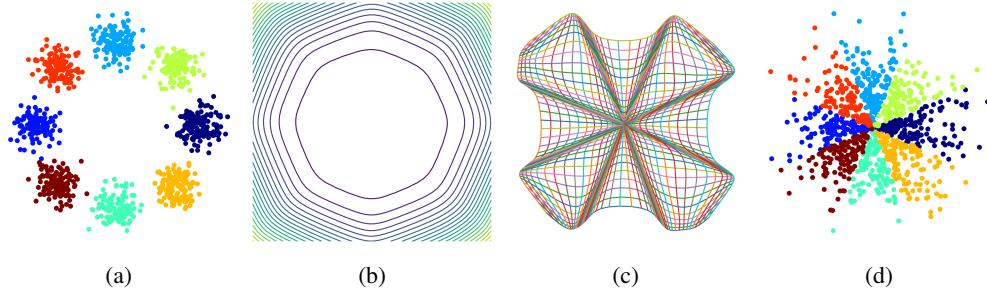


Figure 1: Illustration of Convex Potential Flow. (a) Data x drawn from a mixture of Gaussians. (b) Learned convex potential F . (c) Mesh grid distorted by the gradient map of the convex potential $f = \nabla F$. (d) Encoding of the data via the gradient map $z = f(x)$. Notably, the encoding is the *value of the gradient* of the convex potential. When the curvature of the potential function is locally flat, gradient values are small and this results in a contraction towards the origin.

3 CONVEX POTENTIAL FLOWS

Given a strictly convex potential F , we can define an injective map (invertible from its image) via its gradient $f = \nabla F$, since the Jacobian of f is the Hessian matrix of F , and is thus positive definite. In this section, we discuss the parameterization of the convex potential F (3.1), and then address gradient estimation for CP-Flows (3.2). We examine the connection to other parameterization of normalizing flows (3.3), and finally rigorously prove universality in the next section.

3.1 MODELING

Input Convex Neural Networks We use $L(x)$ to denote a linear layer, and $L^+(x)$ to denote a linear layer with positive weights. We use the (fully) input-convex neural network (ICNN, Amos et al. (2017)) to parameterize the convex potential, which has the following form

$$F(x) = L_{K+1}^+(s(z_K)) + L_{K+1}(x) \quad z_k := L_k^+(s(z_{k-1})) + L_k(x) \quad z_1 := L_1(x)$$

where s is a non-decreasing, convex activation function. In this work, we use softplus-type activation functions, which is a rich family of activation functions that can be shown to uniformly approximate the ReLU activation. See Appendix B for details.

Invertibility and Inversion Procedure If the activation s is twice differentiable, then the Hessian H_F is positive semi-definite. We can make it strongly convex by adding a quadratic term $F_\alpha(x) = \frac{\alpha}{2}\|x\|_2^2 + F(x)$, such that $H_{F_\alpha} \succeq \alpha I \succ 0$. This means the gradient map $f_\alpha = \nabla F_\alpha$ is injective onto its image. Furthermore, it is surjective since for any $y \in \mathbb{R}^d$, the potential $x \mapsto F_\alpha(x) - y^\top x$ has a unique minimizer¹ satisfying the first order condition $\nabla F_\alpha(x) = y$, due to the strong convexity and differentiability. We refer to this invertible mapping f_α as the *convex potential flow*, or the CP-Flow. The above discussion also implies we can plug in a black-box convex solver to invert the gradient map f_α , which we summarize in Algorithm 1. Inverting a batch of independent inputs is as simple as summing the convex potential over all inputs: since all of the entries of the scalar l in the mini-batch are independent of each other, computing the gradient all l 's wrt all x 's amounts to computing the gradient of the summation of l 's wrt all x 's. Due to the convex nature of the problem, a wide selection of algorithms can be used with convergence guarantees (Nesterov, 1998). In practice, we use the L -BFGS algorithm (Byrd et al., 1995) as our `CvxSolver`.

Algorithm 1 Inverting CP-Flow.

```

1: procedure INVERT( $F, y, \text{CvxSolver}$ )
2:   Initialize  $x \leftarrow y$ 
3:   def closure():
4:     Compute loss:  $l \leftarrow F(x) - y^\top x$ 
5:     return  $l$ 
6:    $x \leftarrow \text{CvxSolver}(\text{closure}, x)$ 
7:   return  $x$ 

```

¹The minimizer x^* corresponds to the gradient map of the *convex conjugate* of the potential. See Appendix A for a formal discussion.

Estimating Log Probability Following equation (1), computing the log density for CP-Flows requires taking the log determinant of a symmetric positive definite Jacobian matrix (as it is the Hessian of the potential). There exists numerous works on estimating spectral densities (e.g. Tal-Ezer & Kosloff, 1984; Silver & Röder, 1994; Han et al., 2018; Adams et al., 2018), of which this quantity is a special case. See Lin et al. (2016) for an overview of methods that only require access to Hessian-vector products. Hessian-vector products (hvp) are cheap to compute with reverse-mode automatic differentiation (Baydin et al., 2017), which does not require constructing the full Hessian matrix and has the same asymptotic cost as evaluating F_α .

In particular, the log determinant can be rewritten in the form of a generalized trace $\text{tr} \log H$. Chen et al. (2019a) limit the spectral norm (i.e. eigenvalues) of H and directly use the Taylor expansion of the matrix logarithm. Since our H has unbounded eigenvalues, we use a more complex algorithm designed for symmetric matrices, the *stochastic Lanczos quadrature* (SLQ; Ubaru et al., 2017). At the core of SLQ is the Lanczos method, which computes m eigenvalues of H by first constructing a symmetric tridiagonal matrix $T \in \mathbb{R}^{m \times m}$ and computing the eigenvalues of T . The Lanczos procedure only requires Hessian-vector products, and it can be combined with a stochastic trace estimator to provide a stochastic estimate of our log probability. We chose SLQ because it has shown theoretically and empirically to have low variance (Ubaru et al., 2017).

3.2 $\mathcal{O}(1)$ -MEMORY UNBIASED $\nabla \log \det H$ ESTIMATOR

We would also like to have an estimator for the *gradient* of the log determinant to enable variants of stochastic gradient descent for optimization. Unfortunately, directly backpropagating through the log determinant estimator is not ideal. Two major drawbacks of directly differentiating through SLQ are that it requires (i) differentiating through an eigendecomposition routine and (ii) storing all Hessian-vector products in memory (see fig. 2). Problem (i) is more specific to SLQ, because the gradient of an eigendecomposition is not defined when the eigenvalues are not unique (Seeger et al., 2017). Consequently, we have empirically observed that differentiating through SLQ can be unstable, frequently resulting in NaNs due to the eigendecomposition. Problem (ii) will hold true for other algorithms that also estimate $\log \det H$ with Hessian-vector products, and generally the only difference is that a different numerical routine would need to be differentiated through. Due to these problems, we do not differentiate through SLQ, but we still use it as an efficient method for monitoring training progress.

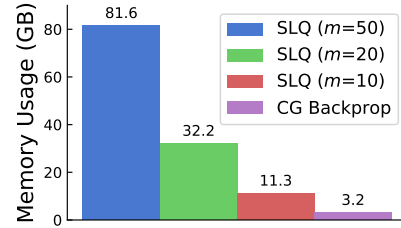


Figure 2: Memory for training CIFAR-10.

Instead, it is possible to construct an alternative formulation of the gradient as the solution of a convex optimization problem, foregoing the necessity of differentiating through an estimation routine of the log determinant. We adapt the gradient formula from Chen et al. (2019a, Appendix C) to the context of convex potentials. Using Jacobi’s formula* and the adjugate representation of the matrix inverse[†], for any invertible matrix H with parameter θ , we have the following identity:

$$\frac{\partial}{\partial \theta} \log \det H = \frac{1}{\det H} \frac{\partial}{\partial \theta} \det H \stackrel{*}{=} \frac{1}{\det H} \text{tr} \left(\text{adj}(H) \frac{\partial H}{\partial \theta} \right) \stackrel{\dagger}{=} \text{tr} \left(H^{-1} \frac{\partial H}{\partial \theta} \right) = \mathbb{E}_v \left[v^\top H^{-1} \frac{\partial H}{\partial \theta} v \right]. \quad (3)$$

Notably, in the last equality, we used the Hutchinson trace estimator (Hutchinson, 1989) with a Rademacher random vector v , leading to a $\mathcal{O}(1)$ -memory, unbiased Monte Carlo gradient estimator.

Computing the quantity $v^\top H^{-1}$ in eq. (3) by constructing and inverting the full Hessian requires d calls to an automatic differentiation routine and is too costly for our purposes. However, we can recast this quantity as the solution of a quadratic optimization problem

$$\arg \min_z \left\{ \frac{1}{2} z^\top H z - v^\top z \right\} \quad (4)$$

which has the unique minimizer $z^* = H^{-1}v$ since H is symmetric positive definite.

We use the *conjugate gradient* (CG) method, which is specifically designed for solving the unconstrained optimization problems in eq. (4) with symmetric positive definite H . It uses only Hessian-vector products and is straightforward to parallelize. Conjugate gradient is guaranteed to return the exact solution z^* within d iterations, and the error of the approximation is known to converge exponentially fast $\|z^m - z^*\|_H \leq 2\gamma^m \|z^0 - z^*\|_H$, where z^m is the estimate after m iterations. The rate of convergence $\gamma < 1$ relates to the condition number of H . For more details, see Nocedal & Wright (2006, Ch. 5). In practice, we terminate CG when $\|Hz^m - v\|_\infty < \tau$ is satisfied for some user-controlled tolerance. Empirically, we find that stringent tolerance values are unnecessary for stochastic optimization (see appendix F).

Estimating the full quantity in eq. (3) is then simply a matter of computing and differentiating a scalar quantity (a surrogate objective) involving another Hessian-vector product: $\frac{d}{d\theta} ((z^m)^\top H v)$, where only H is differentiated through (since z^m is only used to approximate $v^\top H^{-1}$ as a modifier of the gradient). We summarize this procedure in Algorithm 2. Similar to inversion, the hvp can also be computed in batch by summing over the data index, since all entries are independent.

3.3 CONNECTION TO OTHER NORMALIZING FLOWS

Residual Flow For $\alpha = 1$, the gradient map f_1 resembles the residual flow (Behrmann et al., 2019; Chen et al., 2019a). They require the residual block—equivalent to our gradient map f —to be *contractive* (with Lipschitz constant strictly smaller than 1) as a sufficient condition for invertibility. In contrast, we enforce invertibility by using strongly convex potentials, which guarantees that the inverse of our flow is globally unique. With this, we do not pay the extra compute cost for having to satisfy Lipschitz constraints using methods such as spectral normalization (Miyato et al., 2018). Our gradient estimator is also derived similarly to that of Chen et al. (2019a), though we have the benefit of using well-studied convex optimization algorithms for computing the gradients.

Sylvester Flow By restricting the architecture of our ICNN to one hidden layer, we can also recover a form similar to Sylvester Flows. For a 1-hidden layer ICNN ($K = 1$) and $\alpha = 1$, we have $F_1 = \frac{1}{2}\|x\|_2^2 + L_2^+(s(L_1x)) + L_2(x)$. Setting the weights of L_2 to zero, we have

$$f_1(x) = \nabla_x F_1(x) = x + W_1^\top \text{diag}(w_2^+) s'(W_1x + b_1). \quad (5)$$

We notice the above form bears a close resemblance to the Sylvester normalizing flow (Van Den Berg et al., 2018) (with Q , R and $\tilde{R}$ from Van Den Berg et al. (2018) being equal to $W_1^\top$, $\text{diag}(w_2^+)$ and I , respectively). For the Sylvester flow to be invertible, they require that R and $\tilde{R}$ be triangular and Q be orthogonal, which is a computationally costly procedure. This orthogonality constraint also implies that the number of hidden units cannot exceed d . This restriction to orthogonal matrices and one hidden layer are for applying Sylvester’s determinant identity. In contrast, we do not require our weight matrices to be orthogonal, and we can use any hidden width and depth for the ICNN.

Sigmoidal Flow Let s be the softplus activation function and $\sigma = s'$. Then for the 1-dimensional case ($d = 1$) and $\alpha = 0$ (without the residual connection), we have

$$\frac{\partial}{\partial x} F_0(x) = \sum_{j=1} w_{1,j} w_{2,j}^+ \sigma(w_{1,j}x + b_{1,j}) = \sum_{j=1} |w_{1,j}| w_{2,j}^+ \sigma(|w_{1,j}|x + \text{sign}(w_{1,j})b_{1,j}) + \text{const.} \quad (6)$$

which is equivalent to the sigmoidal flow of Huang et al. (2018) up to rescaling (since the weighted sum is no longer a convex sum) and a constant shift, and is monotone due to the positive weights. This correspondence is not surprising since a differentiable function is convex if and only if its derivative is monotonically non-decreasing. It also means we can parameterize an increasing function as the derivative of a convex function, which opens up a new direction for parameterizing autoregressive normalizing flows (Kingma et al., 2016; Huang et al., 2018; Müller et al., 2019; Jaini et al., 2019; Durkan et al., 2019; Wehenkel & Louppe, 2019).

Algorithm 2 Surrogate training objective.

```

1: procedure SURROGATEOBJ( $F, x, \text{CG}$ )
2:   Obtain the gradient  $f(x) \triangleq \nabla_x F(x)$ 
3:   Sample Rademacher random vector  $r$ 
4:   def hvp( $v$ ):
5:     return  $v^\top \frac{\partial}{\partial x} f(x)$ 
6:    $z \leftarrow \text{stop\_gradient}(\text{CG}(\text{hvp}, r))$ 
7:   return  $\text{hvp}(z)^\top r$ 

```

Flows with Potential Parameterization Inspired by connections between optimal transport and continuous normalizing flows, some works (Zhang et al., 2018; Finlay et al., 2020a; Onken et al., 2020) have proposed to parameterize continuous-time transformations by taking the gradient of a scalar potential. They do not strictly require the potential to be convex since it is guaranteed to be invertible in the infinitesimal setting of continuous normalizing flows (Chen et al., 2018). There exist works (Yang & Karniadakis, 2019; Finlay et al., 2020b; Onken et al., 2020) that have applied the theory of optimal transport to regularize continuous-time flows to have low transport cost. In contrast, we connect optimal transport with discrete-time normalizing flows, and CP-Flow is guaranteed by construction to converge pointwise to the optimal mapping between distributions without explicit regularization (see Section 4).

4 THEORETICAL ANALYSES

As explained in Section 2, the parameterization of CP-Flow is inspired by the Brenier potential. So naturally we would hope to show that (1) CP-Flows are distributionally universal, and that (2) the learned invertible map is optimal in the sense of the average squared distance the input travels $\mathbb{E}[||x - f(x)||^2]$. Proofs of statements made in this section can be found in Appendices C and D.

To show (1), our first step is to show that ICNNs can approximate arbitrary convex functions. However, convergence of potential functions does not generally imply convergence of the gradient fields. A classic example is the sequence $F_n = \sin(nx)/\sqrt{n}$ and the corresponding derivatives $f_n = \cos(nx)\sqrt{n}$: $F_n \rightarrow 0$ as $n \rightarrow \infty$ but f_n does not. Fortunately, convexity allows us to control the variation of the gradient map (since the derivative of a convex function is monotone), so our second step of approximation holds.

Theorem 2. *Let $F_n : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable convex functions and $G : \mathbb{R}^d \rightarrow \mathbb{R}$ be a proper convex function. Assume $F_n \rightarrow G$. Then for almost every $x \in \mathbb{R}^d$, G is differentiable and $f_n(x) := \nabla F_n(x) \rightarrow \nabla G(x) =: g(x)$.*

Combining these two steps and Brenier’s theorem, we show that CP-Flow with softplus-type activation function is distributionally universal.

Theorem 3 (Universality). *Given random variables $X \sim \mu$ and $Y \sim \nu$, with μ being absolutely continuous w.r.t. the Lebesgue measure, there exists a sequence of ICNN F_n with a softplus-type activation, such that $\nabla F_n \circ X \rightarrow Y$ in distribution.*

N.B. In the theorem we do not require the second moment to be finite, as for arbitrary random variables we can apply the standard truncation technique and redistribute the probability mass so that the new random variables are almost surely bounded. For probability measures with finite second moments, we indeed use the gradient map of ICNN to approximate the optimal transport map corresponding to the Brenier potential. In the following theorem, we show that the optimal transport map is the only such mapping that we can approximate if we match the distributions.

Theorem 4 (Optimality). *Let G be the Brenier potential of $X \sim \mu$ and $Y \sim \nu$, and let F_n be a convergent sequence of differentiable, convex potentials, such that $\nabla F_n \circ X \rightarrow Y$ in distribution. Then ∇F_n converges almost surely to ∇G .*

The theorem states that in practice, even if we optimize according to some loss that traces the convergence in distribution, our model is still able to recover the optimal transport map, as if we were optimizing according to the transport cost. This allows us to estimate optimal transport maps without solving the constrained optimization in (2). See Seguy et al. (2018) for some potential applications of the optimal transport map, such as domain adaptation or domain translation.

5 EXPERIMENT

We use CP-Flow to perform density estimation (RHS of (1)) and variational inference (LHS of (1)) to assess its approximation capability, and the effectiveness of the proposed gradient estimator. All the details of experiments can be found in Appendix E. Code is available at <https://github.com/CW-Huang/CP-Flow>.

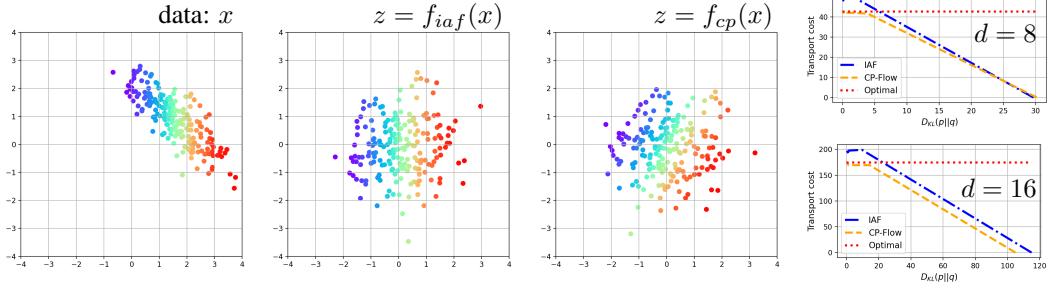


Figure 4: Approximating optimal transport map via maximum likelihood (minimizing KL divergence). In the first figure on the left we show the data in 2 dimensions. The datapoints are colored according to their horizontal values (x_1). The flows f_{iaf} and f_{cp} are trained to transform the data into a standard Gaussian prior. In the figures on the right, we plot the expected quadratic transportation cost versus the KL divergence for different numbers of dimensionality. During training the KL is minimized, so the curves read from the right to the left.

ICNN Architecture Despite the universal property, having a poor parameterization can lead to difficulties in optimization and limit the effective expressivity of the model. We propose an architectural enhancement of ICNN, defined as follows (note the change in notation: instead of writing the pre-activations z , we use h to denote the activated units):

$$F^{aug}(x) := L_{K+1}^+(h_K) + L_{K+1}(x) \\ h_k := \text{concat}([\tilde{h}_k, h_k^{aug}]) \quad \tilde{h}_k := s(L_k^+(h_{k-1}) + L_k(x)) \quad h_k^{aug} = s(L_k^{aug}(x)) \quad (7)$$

where half of the hidden units are directly connected to the input, so the gradient would have some form of skip connection. We call this the input-augmented ICNN. Unless otherwise stated, we use the input-augmented ICNN as the default architecture.

5.1 TOY EXAMPLES

Having distributional universality for a single flow layer means that we can achieve high expressiveness without composing too many flows. We demonstrate this by fitting the density on some toy examples taken from Papamakarios et al. (2017) and Behrmann et al. (2019). We compare with the masked autoregressive flow (MAF, Papamakarios et al. (2017)) and the neural autoregressive flow (NAF, (Huang et al., 2018)). Results are presented in fig. 3. We try to match the network size for each data. All models fit the first data well. As affine couplings cannot split probability mass, MAF fails to fit to the second and third datasets². Although the last dataset is intrinsically harder to fit (as NAF, another universal density model, also fails to fit it well), the proposed method still manages to learn the correct density with high fidelity.

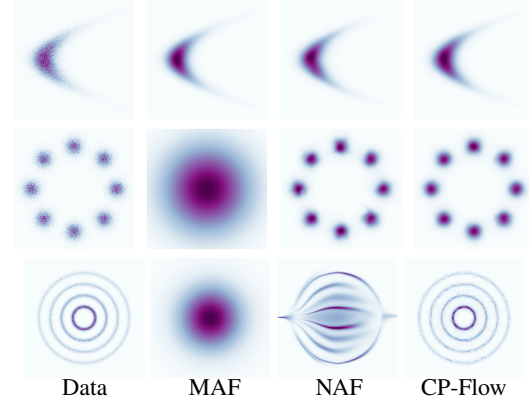


Figure 3: Learning toy densities.

5.2 APPROXIMATING OPTIMAL COUPLING

As predicted by Theorem 4, CP-Flow is guaranteed to converge to the optimal coupling minimizing the expected quadratic cost. We empirically verify it by learning the Gaussian density and comparing the expected quadratic distance between the input and output of the flow against $J_{||x-y||^2}$ between the Gaussian data and the standard Gaussian prior (as there is a closed-form expression). In fig. 4, we see that the transport cost gets closer to the optimal value when the learned density

²Behrmann et al. (2019) demonstrates one can potentially improve the affine coupling models by composing many flow layers. But here we restrict the number of flow layers to be 3 or 5.

Model	POWER	GAS	HEPMASS	MINIBOONE	BSDS300
Real NVP (Dinh et al., 2017)	-0.17	-8.33	18.71	13.55	-153.28
FFJORD (Grathwohl et al., 2018)	-0.46	-8.59	14.92	10.43	-157.40
MADE (Germain et al., 2015)	3.08	-3.56	20.98	15.59	-148.85
MAF (Papamakarios et al., 2017)	-0.24	-10.08	17.70	11.75	-155.69
TAN (Oliva et al., 2018)	-0.48	-11.19	15.12	11.01	-157.03
NAF (Huang et al., 2018)	-0.62	-11.96	15.09	8.86	-157.73
CP-Flow (Ours)	-0.52	-10.36	16.93	10.58	-154.99

Table 1: Average test negative log-likelihood (in nats) of tabular datasets in Papamakarios et al. (2017) for density estimation models (lower is better). Standard deviation is presented in the appendix E.4.

Model	MNIST		CIFAR-10	
	Bits/dim	N. params	Bits/dim	N. params
Real NVP (Dinh et al., 2017)	1.05	N/A	3.49	N/A
Glow (Kingma & Dhariwal, 2018)	1.06	N/A	3.35	44.0M [†]
RQ-NSF (Durkan et al., 2019)	—	—	3.38	11.8M [†]
Residual Flow (Chen et al., 2019a)	0.97	16.6M [‡]	3.28	25.2M [‡]
Coupling Block Ablation	1.02	3.1M	3.58	2.9M
Residual Block Ablation	1.04	2.9M	3.46	3.1M
CP-Flow (Ours)	1.02	2.9M	3.40	1.9M

Table 2: Negative log-likelihood (in bits) on held-out test data (lower is better). [†]Taken from Durkan et al. (2019). [‡]Obtained from official open source code.

approaches the data distribution (measured by the KL divergence). We compare against the linear inverse autoregressive flow (Kingma et al., 2016), which has the capacity to represent the multivariate Gaussian density, yet it does not learn the optimal coupling.

5.3 DENSITY ESTIMATION

We demonstrate the efficacy of our model and the proposed gradient estimator by performing density estimation on the standard benchmarks.

Tabular Data We use the datasets preprocessed by Papamakarios et al. (2017). In table 1, we report average negative log-likelihood estimates evaluated on held-out test sets, for the best hyperparameters found via grid search. The search was focused on the number of flow blocks, the width and depth of the ICNN potentials. See appendix E.4 for details. Our models perform competitively against alternative approaches in the literature. We also perform an ablation on the CG error tolerance and ICNN architectures in appendix F.

Image Data Next, we apply CP-Flow to model the density of standard image datasets, MNIST and CIFAR-10. For this, we use convolutional layers in place of fully connected layers. Prior works have had to use large architectures, with many flow blocks composed together, resulting in a large number of parameters to optimize. While we also compose multiple blocks of CP-Flows, we find that CP-Flow can perform relatively well with fewer number of parameters (table 2). Notably, we achieve comparable bits per dimension to Neural Spline Flows (Durkan et al., 2019)—another work promoting fewer parameters—while having using around 16% number of parameters.

As prior works use different architectures with widely varying hyperparameters, we perform a more careful ablation study using coupling (Dinh et al., 2014; 2017) and invertible residual blocks (Chen et al., 2019a). We replace each of our flow blocks with the corresponding baseline. We find that on CIFAR-10, the baseline flow models do not perform nearly as well as CP-Flow. We believe this may be because CP-Flows are universal with just one flow block, whereas coupling and invertible residual blocks are limited in expressivity or Lipschitz-constrained.

5.4 AMORTIZING ICNN FOR VARIATIONAL INFERENCE

Normalizing flows also allow us to employ a larger, more flexible family of distributions for variational inference (Rezende & Mohamed, 2015). We replicate the experiment conducted in Van Den Berg et al. (2018) to enhance the variational autoencoder (Kingma & Welling, 2013). For inference amortization, we use the partially input convex neural network from Amos et al. (2017), and use the output of the encoder as the additional input for conditioning. As table 3 shows, the performance of CP-Flow is close to the best reported in Van Den Berg et al. (2018) without changing the experiment setup. This shows that the convex potential parameterization along with the proposed gradient estimator can learn to perform accurate amortized inference. Also, we show that replacing the vanilla ICNN with the input-augmented ICNN leads to improvement of the likelihood estimates.

	FREYFACES	OMNIGLOT	CALTECH
Gaussian	4.53	104.28	110.80
Planar	4.40	102.65	109.66
IAF	4.47	102.41	111.58
Sylvester	4.45	99.00	104.62
CP-Flow (vanilla)	4.47	102.06	106.53
CP-Flow (aug)	4.45	100.82	105.17

Table 3: Negative ELBO of VAE (lower is better). Standard deviation reported in appendix E.6.

6 CONCLUSION

We propose a new parameterization of normalizing flows using the gradient map of a convex potential. We make connections to the optimal transport theory to show that the proposed flow is a universal density model, and leverage tools from convex optimization to enable efficient training and model inversion. Experimentally, we show that the proposed method works reasonably well when evaluated on standard benchmarks.

Furthermore, we demonstrate that the performance can be improved by designing better ICNN architectures. We leave the exploration for a better ICNN and convolutional ICNN architecture to improve density estimation and generative modeling for future research.

ACKNOWLEDGEMENTS

We would like to acknowledge the Python community (Van Rossum & Drake Jr, 1995; Oliphant, 2007) for developing the tools that enabled this work, including numpy (Oliphant, 2006; Van Der Walt et al., 2011; Walt et al., 2011; Harris et al., 2020), PyTorch (Paszke et al., 2019), Matplotlib (Hunter, 2007), seaborn (Waskom et al., 2018), pandas (McKinney, 2012), and SciPy (Jones et al., 2014).

REFERENCES

- Ryan P Adams, Jeffrey Pennington, Matthew J Johnson, Jamie Smith, Yaniv Ovadia, Brian Patton, and James Saunderson. Estimating the spectral density of large implicit matrices. *arXiv preprint arXiv:1802.03451*, 2018.
- Brandon Amos, Lei Xu, and J Zico Kolter. Input convex neural networks. In *International Conference on Machine Learning*, pp. 146–155, 2017.
- Atılım Günes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *The Journal of Machine Learning Research*, 18(1):5595–5637, 2017.
- Jens Behrmann, Will Grathwohl, Ricky T. Q. Chen, David Duvenaud, and Jörn-Henrik Jacobsen. Invertible residual networks. In *International Conference on Machine Learning*, pp. 573–582, 2019.
- Jens Behrmann, Paul Vicol, Kuan-Chieh Wang, Roger Grosse, and Jörn-Henrik Jacobsen. Understanding and mitigating exploding inverses in invertible neural networks. *arXiv preprint arXiv:2006.09347*, 2020.

- Yann Brenier. Décomposition polaire et réarrangement monotone des champs de vecteurs. *CR Acad. Sci. Paris Sér. I Math.*, 305:805–808, 1987.
- Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on pure and applied mathematics*, 44(4):375–417, 1991.
- Richard H Byrd, Peihuang Lu, Jorge Nocedal, and Ciyu Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing*, 16(5):1190–1208, 1995.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In *Advances in neural information processing systems*, pp. 6571–6583, 2018.
- Ricky T. Q. Chen, Jens Behrmann, David K Duvenaud, and Jörn-Henrik Jacobsen. Residual flows for invertible generative modeling. In *Advances in Neural Information Processing Systems*, pp. 9916–9926, 2019a.
- Yize Chen, Yuanyuan Shi, and Baosen Zhang. Optimal control via neural networks: A convex approach. In *International Conference on Learning Representations*, 2019b.
- Mircea Cimpoi, Subhansu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Conference on Computer Vision and Pattern Recognition*, 2014.
- Taco Cohen and Max Welling. Group equivariant convolutional networks. In *International conference on machine learning*, pp. 2990–2999, 2016.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. In *International Conference on Learning Representations*, 2017.
- Vincent Dumoulin, Ethan Perez, Nathan Schucher, Florian Strub, Harm de Vries, Aaron Courville, and Yoshua Bengio. Feature-wise transformations. *Distill*, 3(7):e11, 2018.
- Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows. In *Advances in Neural Information Processing Systems*, pp. 7511–7522, 2019.
- Chris Finlay, Augusto Gerolin, Adam M Oberman, and Aram-Alexandre Pooladian. Learning normalizing flows from entropy-kantorovich potentials. *arXiv preprint arXiv:2006.06033*, 2020a.
- Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M Oberman. How to train your neural ode: the world of jacobian and kinetic regularization. In *International Conference on Machine Learning*, 2020b.
- Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. Made: Masked autoencoder for distribution estimation. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 881–889. PMLR, 07–09 Jul 2015.
- Ian Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *International conference on machine learning*, pp. 1319–1327. PMLR, 2013.
- Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.
- Insu Han, Haim Avron, and Jinwoo Shin. Stochastic chebyshev gradient descent for spectral optimization. In *Advances in Neural Information Processing Systems*, pp. 7386–7396, 2018.

- Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- Chin-Wei Huang, David Krueger, Alexandre Lacoste, and Aaron Courville. Neural autoregressive flows. In *International Conference on Machine Learning*, pp. 2078–2087, 2018.
- Chin-Wei Huang, Laurent Dinh, and Aaron Courville. Augmented normalizing flows: Bridging the gap between generative flows and latent variable models. *arXiv preprint arXiv:2002.07101*, 2020a.
- Chin-Wei Huang, Laurent Dinh, and Aaron Courville. Solving ode with universal flows: Approximation theory for flow-based models. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020b.
- Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4700–4708, 2017.
- John D Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(3): 90, 2007.
- Michael F Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- Priyank Jaini, Kira A Selby, and Yaoliang Yu. Sum-of-squares polynomial flow. In *International Conference on Machine Learning*, pp. 3009–3018, 2019.
- Eric Jones, Travis Oliphant, and Pearu Peterson. {SciPy}: Open source scientific tools for {Python}. 2014.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *Advances in neural information processing systems*, pp. 10215–10224, 2018.
- Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Advances in neural information processing systems*, pp. 4743–4751, 2016.
- Zhifeng Kong and Kamalika Chaudhuri. The expressive power of a class of normalizing flow models. *arXiv preprint arXiv:2006.00392*, 2020.
- Alexander Korotin, Vage Egiazarian, Arip Asadulaev, Alexander Safin, and Evgeny Burnaev. Wasserstein-2 generative networks. *arXiv preprint arXiv:1909.13082*, 2019.
- Hongzhou Lin and Stefanie Jegelka. Resnet with one-neuron hidden layers is a universal approximator. In *Advances in neural information processing systems*, pp. 6169–6178, 2018.
- Lin Lin, Yousef Saad, and Chao Yang. Approximating spectral densities of large matrices. *SIAM review*, 58(1):34–65, 2016.
- Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, and Liwei Wang. The expressive power of neural networks: A view from the width. In *Advances in neural information processing systems*, pp. 6231–6239, 2017.
- Ashok Vardhan Makkuva, Amirhossein Taghvaei, Sewoong Oh, and Jason D Lee. Optimal transport mapping via input convex neural networks. *arXiv preprint arXiv:1908.10962*, 2019.
- Wes McKinney. *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython*. ”O’Reilly Media, Inc.”, 2012.

- Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *arXiv preprint arXiv:1802.05957*, 2018.
- Thomas Müller, Brian McWilliams, Fabrice Rousselle, Markus Gross, and Jan Novák. Neural importance sampling. *ACM Transactions on Graphics (TOG)*, 38(5):1–19, 2019.
- Yurii Nesterov. Introductory lectures on convex programming volume i: Basic course. *Lecture notes*, 3(4):5, 1998.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011.
- Jorge Nocedal and Stephen Wright. *Numerical optimization*. Springer Science & Business Media, 2006.
- Travis E Oliphant. *A guide to NumPy*, volume 1. Trelgol Publishing USA, 2006.
- Travis E Oliphant. Python for scientific computing. *Computing in Science & Engineering*, 9(3): 10–20, 2007.
- Junier Oliva, Avinava Dubey, Manzil Zaheer, Barnabas Poczos, Ruslan Salakhutdinov, Eric Xing, and Jeff Schneider. Transformation autoregressive networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 3898–3907. PMLR, 10–15 Jul 2018.
- Derek Onken, Samy Wu Fung, Xingjian Li, and Lars Ruthotto. Ot-flow: Fast and accurate continuous normalizing flows via optimal transport. *arXiv preprint arXiv:2006.00104*, 2020.
- George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. In *Advances in Neural Information Processing Systems*, pp. 2338–2347, 2017.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, pp. 8026–8037, 2019.
- Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning*, pp. 1530–1538, 2015.
- R Tyrrell Rockafellar. *Convex analysis*. Number 28. Princeton university press, 1970.
- Ludger Rüschendorf and Svetlozar T Rachev. A characterization of random variables with minimum 12-distance. *Journal of multivariate analysis*, 32(1):48–54, 1990.
- Filippo Santambrogio. Optimal transport for applied mathematicians. *Birkhäuser, NY*, 55(58-63):94, 2015.
- Matthias Seeger, Asmus Hetzel, Zhenwen Dai, Eric Meissner, and Neil D Lawrence. Auto-differentiating linear algebra. *arXiv preprint arXiv:1710.08717*, 2017.
- Vivien Seguy, Bharath Bhushan Damodaran, Remi Flamary, Nicolas Courty, Antoine Rolet, and Mathieu Blondel. Large scale optimal transport and mapping estimation. In *International Conference on Learning Representations*, 2018.
- RN Silver and H Röder. Densities of states of mega-dimensional hamiltonian matrices. *International Journal of Modern Physics C*, 5(04):735–753, 1994.
- Amirhossein Taghvaei and Amin Jalali. 2-wasserstein approximation via restricted convex potentials with application to improved training for gans. *arXiv preprint arXiv:1902.07197*, 2019.
- Hillel Tal-Ezer and Ronnie Kosloff. An accurate and efficient scheme for propagating the time dependent schrödinger equation. *The Journal of chemical physics*, 81(9):3967–3971, 1984.

- Takeshi Teshima, Isao Ishikawa, Koichi Tojo, Kenta Oono, Masahiro Ikeda, and Masashi Sugiyama. Coupling-based invertible neural networks are universal diffeomorphism approximators. *arXiv preprint arXiv:2006.11469*, 2020.
- Shashanka Ubaru, Jie Chen, and Yousef Saad. Fast estimation of $\text{tr}(f(A))$ via Stochastic Lanczos Quadrature. *SIAM Journal on Matrix Analysis and Applications*, 38(4):1075–1099, 2017.
- Rianne Van Den Berg, Leonard Hasenclever, Jakub M Tomczak, and Max Welling. Sylvester normalizing flows for variational inference. In *34th Conference on Uncertainty in Artificial Intelligence 2018, UAI 2018*, pp. 393–402. Association For Uncertainty in Artificial Intelligence (AUAI), 2018.
- Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. The numpy array: a structure for efficient numerical computation. *Computing in Science & Engineering*, 13(2):22, 2011.
- Guido Van Rossum and Fred L Drake Jr. *Python reference manual*. Centrum voor Wiskunde en Informatica Amsterdam, 1995.
- Cédric Villani. *Optimal transport: old and new*, volume 338. Springer Science & Business Media, 2008.
- Stéfan van der Walt, S Chris Colbert, and Gael Varoquaux. The numpy array: a structure for efficient numerical computation. *Computing in science & engineering*, 13(2):22–30, 2011.
- Michael Waskom, Olga Botvinnik, Drew O’Kane, Paul Hobson, Joel Ostblom, Saulius Lukauskas, David C Gemperline, Tom Augspurger, Yaroslav Halchenko, John B. Cole, Jordi Warmenhoven, Julian de Ruiter, Cameron Pye, Stephan Hoyer, Jake Vanderplas, Santi Villalba, Gero Kunter, Eric Quintero, Pete Bachant, Marcel Martin, Kyle Meyer, Alistair Miles, Yoav Ram, Thomas Brunner, Tal Yarkoni, Mike Lee Williams, Constantine Evans, Clark Fitzgerald, Brian, and Adel Qalieh. mwaskom/seaborn: v0.9.0 (july 2018), July 2018. URL <https://doi.org/10.5281/zenodo.1313201>.
- Antoine Wehenkel and Gilles Louppe. Unconstrained monotonic neural networks. In *Advances in Neural Information Processing Systems*, pp. 1545–1555, 2019.
- Liu Yang and George Em Karniadakis. Potential flow generator with l_2 optimal transport regularity for generative models. *arXiv preprint arXiv:1908.11462*, 2019.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. Deep sets. In *Advances in neural information processing systems*, pp. 3391–3401, 2017.
- Linfeng Zhang, Lei Wang, et al. Monge-ampère flow for generative modeling. *arXiv preprint arXiv:1809.10188*, 2018.
- Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

A INVERTIBILITY OF CP-FLOW

In this section, we formally discuss the invertibility of CP-Flow, and establish the connection to convex conjugate (Legendre-Fenchel transform). We work with C^2 convex potentials $F : \mathbb{R}^d \rightarrow \mathbb{R}$; i.e. F is convex and twice continuously differentiable. We first check that $f := \nabla F$ is injective if F is *strictly* convex. This is because if F is twice differentiable and strictly convex, the Hessian matrix $H := \nabla^2 F$ is symmetric positive definite, and thus $z^\top H z > 0$ for any non-zero vector z . We then have, for any $x \neq y$,

$$f(x) - f(y) = \int_{\gamma} H(\gamma) d\gamma = \int_0^1 H(y + t(x - y))(x - y) dt,$$

where we used the *gradient theorem* for the line integral on a path γ connecting x and y , and substituted $t \mapsto y + t(x - y)$ for t going from 0 to 1. Positive-definiteness implies $(x - y)^\top (f(x) - f(y)) > 0$, and since $x \neq y$, $f(x) \neq f(y)$.

Now we further assume F is *strongly* convex. Then for any y , $F_y(x) := F(x) - x^\top y$ is also strongly convex, which, by Taylor's theorem, implies that we can place a quadratic lower bound on F_y and thus $F_y(x) \rightarrow \infty$ whenever $\|x\| \rightarrow \infty$. This means for a sufficiently large constant R , the sub-level set $S_R := \{x : F_y(x) \leq R\}$ is non-empty and compact. By the *Weierstrass extreme value theorem*, F_y (restricted on S_R) has a minimizer x^* , and it is also the global minimizer over $\mathbb{R}^d$. Now let's differentiate F_y at x^* , which gives $\nabla F(x^*) - y$. The gradient must be equal to 0 by the first order condition, meaning x^* is the inverse point of y under f . Since this holds for any $y \in \mathbb{R}^d$, f is surjective.

Now recall the definition of the convex conjugate:

$$F^*(y) := \sup_x x^\top y - F(x) = x^*{}^\top y - F(x^*),$$

where x^* found by the above procedure depends on y . Note that x^* is differentiable by the inverse function theorem. Thus, differentiating F^* yields

$$\nabla_y F^*(y) = (\nabla_y x^*)^\top y + x^* - (\nabla_y x^*)^\top \nabla F(x^*) = x^*$$

since $\nabla F(x^*) = y$. This means if $y = f(x) = \nabla_x F(x)$, then $x = \nabla_y F^*(y)$; i.e. $\nabla F^* = (\nabla F)^{-1}$.

B SOFTPLUS TYPE ACTIVATION

In this section, we let $r(x) = \max(0, x)$ be the ReLU activation function.

Definition 1. We say a function s is of the softplus type if the following holds

- (a) $s \geq r$
- (b) s is convex
- (c) $|s(x) - r(x)| \rightarrow 0$ as $|x| \rightarrow \infty$

Note that a softplus-type activation function is necessarily continuous, non-decreasing, and uniformly approximating ReLU in the following sense:

$$|s(xa)/a - r(x)| \rightarrow 0$$

uniformly for all $x \in \mathbb{R}$ as $a \rightarrow \infty$.

The following proposition characterizes a big family of softplus-type functions, and establishes a close connection between softplus type functions and probability distribution functions.

Proposition 1. Let p be a probability density function of a random variable with mean zero. Then the convolution $s := p * r$ is a softplus-type function. Moreover, $s(x) = \int_{-\infty}^x F_p(y) dy$, where F_p is the distribution function of p , and s is at least twice differentiable.

Proof. We first prove a claim (i): $xF_p(x) \rightarrow 0$ as $x \rightarrow -\infty$. First, for $x \leq 0$,

$$0 \geq xF_p(x) = \int_{-\infty}^x xp(y)dy \geq \int_{-\infty}^x yp(y)dy$$

Since $1_{y \leq x}yp(y) \rightarrow 0$ as $x \rightarrow -\infty$ and $|1_{y \leq x}yp(y)| \leq |yp(y)|$, which is integrable by assumption, the integral on the RHS of the above goes to 0 by the dominated convergence theorem.

We now show the identity. By definition, since $\int yp(y)dy = 0$

$$\begin{aligned} s(x) &= \int \max(x - y, 0)p(y)dy = \int \max(x, y)p(y)dy \\ &= \int_{-\infty}^x xp(y)dy + \int_x^{\infty} yp(y)dy = xF_p(x) + \int_x^{\infty} yp(y)dy \end{aligned} \quad (8)$$

where we've used claim (i) to evaluate $xF_p(x)$ as $x \rightarrow -\infty$. On the other hand, integration by part implies

$$\int_{-\infty}^x F_p(y)dy = yF_p(y)|_{-\infty}^x - \int_{-\infty}^x yp(y)dy = xF_p(x) + \int_x^{\infty} yp(y)dy \quad (9)$$

Twice differentiability follows from the differentiability of F_p .

(a) Now since $r(x)$ is convex, Jensen's inequality gives

$$s(x) = \int r(y)p(x - y)dy = \mathbb{E}[r(y)] \geq r(\mathbb{E}[y]) = r(x)$$

(b) s is convex because $s' = F_p$ is non-decreasing.

(c) To show that s and r are asymptotically the same, we notice the integral on the RHS of (8) goes to 0 as $|x| \rightarrow \infty$ (by the dominated or monotone convergence theorem). It suffices to show $xF_p(x)$ goes to 0 as $x \rightarrow -\infty$, which is just claim (i), and $x - xF_p(x) > 0$ goes to 0 as $x \rightarrow \infty$. To show the latter, we can rewrite $x - xF_p(x) = x(1 - F_p(x)) = \int_x^{\infty} xp(y)dy$, and the same argument as the claim holds with a vanishing upper bound. $\square$

When p is taken to be the standard logistic density, the corresponding $s = p * r$ is simply the regular softplus activation function. We list a few other softplus-type functions in table 4 and visualize them in fig. 5. We experimented with the Gaussian-softplus and the logistic-softplus.

	$p(y)$	$s := p * r$
Logistic	$\frac{\exp(-x)}{(1+\exp(-x))^2}$	$\log(1 + \exp(x))$
Laplace	$\frac{e^{- x }}{2}$	$r(x) + \frac{e^{- x }}{2}$
Gaussian	$\frac{e^{-\frac{x^2}{2}}}{\sqrt{2\pi}}$	$\sqrt{\frac{\pi}{2}}x \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) + e^{-\frac{x^2}{2}} + \sqrt{\frac{\pi}{2}}x$

Table 4: Formula of some softplus-type functions.

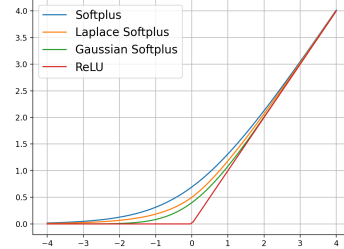


Figure 5: Softplus-type functions.

C UNIVERSALITY PROOF

Notation: Given a convex set $\Omega \subseteq \mathbb{R}^d$, we let $\mathcal{C}(\Omega)$ denote the set of continuous functions on Ω , and $\mathcal{C}_\times(\Omega) := \{f \in \mathcal{C}(\Omega) : f \text{ is convex}\}$ denote the set of convex, continuous functions.

We first show that ICNNs with a suitable activation function are dense in $\mathcal{C}_\times$. A similar result can be found in [Chen et al. \(2019b\)](#), where they use a different constructive proof: first show that piecewise maximum of affine functions, *i.e.* the maxout unit ([Goodfellow et al., 2013](#)), can approximate any convex function, and then represent maxout using ICNN. We emphasize our construction is simpler (see proof of Proposition 3).

The following proposition proves that functions that are pointwise maximum of affine functions, are a dense subset of $\mathcal{C}_\times$.

Proposition 2. *Pointwise maximum of affine functions is dense in $\mathcal{C}_\times([0, 1]^d)$.*

Proof. Fix some $\epsilon > 0$. Since $f \in \mathcal{C}_\times([0, 1]^d)$ is uniformly continuous on $[0, 1]^d$, there exists some $\delta > 0$ such that $|f(x) - f(y)| < \epsilon$ provided that $\|x - y\| < \delta$. Let n be big enough such that $2^{-n} < \delta$, and let $\mathcal{X}$ be the set of points whose coordinates sit on $i2^{-n}$ for some $1 \leq i \leq 2^n - 1$ (*i.e.* there are $|\mathcal{X}| = (2^n - 1)^d$ points in $\mathcal{X}$). For each $y \in \mathcal{X}$, let $L_y(x) := \nabla f(y)^\top (x - y) + f(y)$ be a supporting hyperplane of the graph of f , where $\nabla f(y)$ is a subgradient of f evaluated at y . Then we have a convex approximation $f_\epsilon(x) := \max_{y \in \mathcal{X}} L_y(x)$ which bounds f from below. Moreover, letting $y_{|x} := \arg \min_{y \in \mathcal{X}} \|x - y\|$, we have (for $x \notin \mathcal{X}$),

$$\begin{aligned} f(x) - f_\epsilon(x) &= f(x) - \max_{y \in \mathcal{X}} L_y(x) \\ &\leq f(x) - L_{y_{|x}}(x) \\ &= f(x) - f(y_{|x}) - \sum_{i=1}^d \nabla f(y_{|x})_i (x_i - y_{|x,i}) \\ &\leq f(x) - f(y_{|x}) + \sum_{i=1}^d |\nabla f(y_{|x})_i| \cdot |x_i - y_{|x,i}| \\ &\leq f(x) - f(y_{|x}) + \sum_{i=1}^d \frac{\epsilon}{|x_i - y_{|x,i}|} \cdot |x_i - y_{|x,i}| \\ &\leq (d+1)\epsilon \end{aligned}$$

Since ϵ is arbitrary, this construction forms a sequence of approximations converging uniformly to f from below. $\square$

The following proposition shows that maxout units can be equivalently represented by ICNN with the ReLU activation, and thus entails the density of the latter (as well as ICNN with softplus activation).

Proposition 3. *ICNN with ReLU or softplus-type activation is dense in $\mathcal{C}_\times([0, 1]^d)$.*

Proof. Let $r(x) = \max(0, x)$ be the ReLU activation function. Any convex piecewise linear function $f(x)$ can be represented by $f(x) = \max(L_1, \dots, L_k)$ where $L_j = a_j^\top x + b_j$, which can then be reduced to

$$\begin{aligned} f(x) &= r(\max(L_1 - L_k, \dots, L_{k-1} - L_k)) + L_k \\ &= r(r(\max(L_1 - L_{k-1}, \dots, L_{k-2} - L_{k-1})) + L_{k-1} - L_k) + L_k \\ &= z_k \end{aligned}$$

where $z_j := r(z_{j-1}) + L'_j$ for $2 \leq j \leq k$, $z_1 = L_1 - L_2$, $L'_j := L_j - L_{j+1}$ for $2 \leq j \leq k-1$, and $L'_k := L_k$.

Since by Proposition 2, pointwise maximum of affine functions is dense in $\mathcal{C}_\times$, so is ICNN with the ReLU activation function. The same holds for softplus since softplus can be used to uniformly approximate ReLU.

□

Theorem 2. *Let $F_n : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable convex functions and $G : \mathbb{R}^d \rightarrow \mathbb{R}$ be a proper convex function. Assume $F_n \rightarrow G$. Then for almost every $x \in \mathbb{R}^d$, G is differentiable and $f_n(x) := \nabla F_n(x) \rightarrow \nabla G(x) =: g(x)$.*

Proof. We let x be a differentiable point of G . Since convergence of derivatives wrt each coordinate can be dealt with independently, we assume $d = 1$ without loss of generality. We can write f_n as

$$f_n(x) = \lim_m f_{nm}(x) \quad \text{where} \quad f_{nm}(x) = \frac{F_n(x - 1/m) - F_n(x)}{-1/m}$$

The problem can be rephrased as proving ³

$$\lim_n \lim_m f_{nm} = \lim_m \lim_n f_{nm} \quad (10)$$

Note that f_{nm} is non-decreasing in m since F_n is convex, and thus $f_{nm} \leq f_n$. Since f_{nm} converges to f_n , for any $\epsilon > 0$, we can find an integer $\mu(\epsilon, n)$ such that for all $m \geq \mu(\epsilon, n)$, $|f_{nm} - f_n| \leq \epsilon$. Let m_k be a subsequence of $\{m \geq 1\}$ defined as $m_k = \mu(2^{-k}, n)$.

Then $|f_{nm_{k+1}} - f_{nm_k}| \leq 2^{-k}$, which is integrable wrt the counting measure on positive integers k , since

$$\int 2^{-k} = \lim_{K \rightarrow \infty} \sum_{k=1}^K 2^{-k} = 1$$

Thus, letting $f_{nm_0} = 0$, by the Dominated Convergence Theorem, we have

$$\lim_n \lim_K f_{nm_K} = \lim_n \int f_{nm_K} - f_{nm_{K-1}} = \int \lim_n f_{nm_K} - f_{nm_{K-1}} = \lim_K \lim_n f_{nm_K}$$

Although we are only looking at the limit of the subsequence m_k , this is sufficient for (10), since the LHS is equal to $\lim_n f_n$ (since each F_n is differentiable), and by linearity of the limit, the RHS is equal to $\lim_K \frac{G(x-1/m_K) - G(x)}{-1/m_K} = g(x)$ (since G is differentiable at x by assumption).

Since the set of points over which G is not differentiable is a set of measure zero (Rockafellar, 1970, Thm. 25.5), the convergence holds almost everywhere. □

Theorem 3 (Universality). *Given random variables $X \sim \mu$ and $Y \sim \nu$, with μ being absolutely continuous w.r.t. the Lebesgue measure, there exists a sequence of ICNN F_n with a softplus-type activation, such that $\nabla F_n \circ X \rightarrow Y$ in distribution.*

Proof. Assume μ and ν have finite second moments. Since μ is absolutely continuous, by **Brenier's theorem**, there exists a convex function $G : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\nabla G(X) \stackrel{d}{=} \nu$ (where the gradient is unique up to changes on a null set). By Proposition 3, there exists a sequence of ICNN F_n converging to G pointwise everywhere. Such a sequence can be found since we can let F_n approximate G with a uniform error of $1/n$ on a compact domain $[-n, n]^d$. Theorem 2 then implies the gradient map $f_n := \nabla F_n$ converges to ∇G pointwise almost everywhere. This implies the weak convergence of the pushforward measure of $f_n \circ X$.

Now remove the finite second moment assumption and let X and Y be random variables distributed according to μ (with Lebesgue density p) and ν , respectively. Denote by B_k a ball of radius $k > 0$ centered at the origin, i.e. $B_k := \{x : \|x\| \leq k\}$. Let $X_k = X1_{X \in B_k} + U_k1_{X \notin B_k}$, where U_k is an independent random variable distributed uniformly on B_k , and let μ_k be the law of X_k . Then $X_k \rightarrow X$ almost surely as $k \rightarrow \infty$, and μ_k is still absolutely continuous wrt the Lebesgue

³Note that there is an implicit dependency on x since the result is pointwise.

measure, with its density being $p(x) + \frac{1}{\text{vol}(B_k)}\mu(\|X\| > k)$ if $\|x\| \leq k$ or 0 otherwise. Let Y_k and ν_k be defined similarly (while ν_k may not be absolutely continuous wrt Lebesgue). From above, since X_k and Y_k are bounded and admit a finite second moment, we know fixing k , we can find a sequence of $f_{k,n} = \nabla F_{k,n}$ such that $f_{k,n} \circ X_k \rightarrow Y_k$ in distribution as $n \rightarrow \infty$. Now since weak convergence is metrizable, choose n_k to be large enough such that the distance between the pushforward of $f_{k,n_k} \circ X_k$ and ν_k is at most $1/k$. An application of triangle inequality of the weak metric implies $f_{k,n_k} \circ X_k \rightarrow Y$ in distribution as $k \rightarrow \infty$. Finally, note that since $\{\|f_{k,n_k} \circ X_k - f_{k,n_k} \circ X\| > \epsilon\} \subseteq \{\|X\| > k\}$, by monotonicity, we have

$$\mathbb{P}\left(\limsup_k \|f_{k,n_k} \circ X_k - f_{k,n_k} \circ X\| > \epsilon\right) \leq \mathbb{P}\left(\limsup_k \|X\| > k\right) = 0 \quad (11)$$

and thus $\|f_{k,n_k} \circ X_k - f_{k,n_k} \circ X\| \rightarrow 0$ almost surely (where $\mathbb{P}$ is the underlying probability measure of the measure space). By Lemma 1 of [Huang et al. \(2020b\)](#) (or equivalently Lemma 2 of [Huang et al. \(2020a\)](#)), $f_{k,n_k} \circ X$ converges in distribution to Y .

□

D OPTIMALITY PROOF

Theorem 4 (Optimality). *Let G be the Brenier potential of $X \sim \mu$ and $Y \sim \nu$, and let F_n be a convergent sequence of differentiable, convex potentials, such that $\nabla F_n \circ X \rightarrow Y$ in distribution. Then ∇F_n converges almost surely to ∇G .*

The result can be deduced from the fact that optimality is “stable” under weak limit; see for example, [Santambrogio \(2015, Thm 1.50\)](#). We prove the special case of quadratic cost function.

Proof. We claim that if F is a convex potential such that $Z = \nabla F(X)$ has ν as its law, then $\nabla F \equiv \nabla G$ almost surely. The proof of the claim is originally due to [Rüschendorf & Rachev \(1990\)](#), but we present it here for completeness. Let Z' be another random variable distributed by ν . Then by the Fenchel-Young inequality (applied to the convex potential F),

$$\mathbb{E}[X^\top Z'] \leq \mathbb{E}[F(X) + F^*(Z')] = \mathbb{E}[F(X) + F^*(Z)] = \mathbb{E}[F(X) + F^*(\nabla F(X))] = \mathbb{E}[X^\top \nabla F(X)]$$

This concludes the proof since ∇G uniquely solves the transportation problem, which is equivalent to finding a transport map $\tilde{g}$ that maximizes the covariance:

$$\mathbb{E}[\|X - \tilde{g}(X)\|^2] = \mathbb{E}[\|X\|^2 + \|\tilde{g}(X)\|^2] - 2\mathbb{E}[X^\top \tilde{g}(X)]$$

Let F_∞ to be the pointwise limit of F_n . Then for any x_1, x_2 and $t \in [0, 1]$,

$$\begin{aligned} F_\infty(tx_1 + (1-t)x_2) &= \lim_{n \rightarrow \infty} F_n(tx_1 + (1-t)x_2) \\ &\leq \lim_{n \rightarrow \infty} tF_n(x_1) + (1-t)F_n(x_2) \\ &= \lim_{n \rightarrow \infty} tF_n(x_1) + \lim_{n \rightarrow \infty} (1-t)F_n(x_2) = tF_\infty(x_1) + (1-t)F_\infty(x_2) \end{aligned}$$

That is, F_∞ is convex. Now since F_n is a convergent sequence of convex functions, its gradient ∇F_n also converges pointwise almost everywhere to ∇F_∞ by Theorem 2. Let ρ denote the Prokhorov metric, which metrizes the weak convergence, and by abuse of notation, we write $\rho(X, Y)$ to denote the distance between the law of X and Y . Then

$$\rho(\nabla F_\infty(X), Y) \leq \rho(\nabla F_\infty(X), \nabla F_n(X)) + \rho(\nabla F_n(X), Y)$$

which means $\nabla F_\infty(X)$ and Y have the same law, ν . Then by the claim, $\nabla F_\infty \equiv \nabla G$, and thus $\nabla F_n \rightarrow \nabla G$ a.s. as $n \rightarrow \infty$.

□

E EXPERIMENTAL DETAILS

E.1 ARCHITECTURE DETAILS

Initialization As ICNNs have positive weights, its initialization has a different dynamics than a standard feed-forward network. If not stated otherwise, all parameters are initialized using standard PyTorch modules (Paszke et al., 2019). To parameterize positive weights, we modify the weight parameters of a standard linear layer with the softplus activation. We then divide all the weights by the total number of incoming units, so that the average magnitude of each hidden unit will not grow as the dimensionality of the previous layer increases.

In addition, we reparameterize the CP-Flow F_α as F_{w_0, w_1} defined as

$$F_{w_0, w_1} = s(w_0)||x||^2/2 + s(w_1)F(x)$$

where s is the regular (logistic) softplus. w_0 is initialized to be $s^{-1}(1)$, and w_1 is initialized to be 0 so F_{w_0, w_1} is closer to the identity map.

Finally, we insert the ActNorm layer (Kingma & Dhariwal, 2018) everywhere before an activation function is applied with the data-dependent initialization.

Activation function We use the following convexity-preserving operators when designing an ICNN:

1. invariance under affine maps: $g \circ f$ is convex if f is linear and g is convex
2. non-negative weighted sums: $\sum_j w_j f_j$ is convex if w_j s are non-negative and f_j s are convex
3. composing with non-decreasing convex functions: $g \circ f$ is convex if f and g are both convex and g is non-decreasing

Notably, in 1., we do not require g to be non-decreasing. We experiment with a symmetrized version of softplus $g(x) = s(x) - 0.5x$ where s is a softplus-type activation, whenever g is used as the first activation. This way, the derivative of g is $s'(x) - 0.5$, which ranges between ± 0.5 and behaves more like tanh (than sigmoid). This can be used for the first hidden layer of a regular ICNN, or the augmented layer of the input-augmented ICNN.

We also experiment with an offset version of softplus, which is defined as $g(x) = s(x) - s(0)$. This way the output of the softplus is more symmetric since it can be negative.

E.2 TOY EXAMPLES

For toy examples, we compute the log-determinant of the Jacobian in a brute-force manner. We use the Adam optimizer with an initial learning rate of 0.005. We create a data set following the toy distribution of size 50000, and train each model for 50 epochs with a minibatch size of 128. For MAF and NAF we cap the gradient norm to be 10 for stability. For CP-Flow, we use the Gaussian softplus as activation, and symmetrize it at the first layers.

Data	N. FLOWS	N. HIDDEN LAYERS	N. HIDDEN UNITS
One moon	5	3	32
Eight Gaussians	5	3	32
Rings	5	5	256

Table 5: Architectural details for toy density estimation.

E.3 APPROXIMATING OPTIMAL COUPLING

For the OT map approximation experiment, we simulate data (of size 50,000) from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$ with a prior $\mu \sim \mathcal{N}(0, I)$ and $\Sigma \sim \mathcal{W}(I, d + 1)$, where $\mathcal{W}$ is the Wishart

distribution. For CP-Flow, we use a network of 5 hidden layers of 64 hidden units, with the Gaussian-softplus and zero-offset. We use the Adam optimizer with a minibatch size of 128, trained for two epochs to generate the figures.

E.4 DENSITY ESTIMATION

Model	POWER	GAS	HEPMASS	MINIBOONE	BSDS300
Real NVP	-0.17 ± 0.01	-8.33 ± 0.14	18.71 ± 0.02	13.55 ± 0.49	-153.28 ± 1.78
Glow	-0.17 ± 0.01	-8.15 ± 0.40	18.92 ± 0.08	11.35 ± 0.07	-155.07 ± 0.03
FFJORD	-0.46 ± 0.01	-8.59 ± 0.12	14.92 ± 0.08	10.43 ± 0.04	-157.40 ± 0.19
MADE	3.08 ± 0.03	-3.56 ± 0.04	20.98 ± 0.02	15.59 ± 0.50	-148.85 ± 0.28
MAF	-0.24 ± 0.01	-10.08 ± 0.02	17.70 ± 0.02	11.75 ± 0.44	-155.69 ± 0.28
TAN	-0.48 ± 0.01	-11.19 ± 0.02	15.12 ± 0.02	11.01 ± 0.48	-157.03 ± 0.07
NAF	-0.62 ± 0.01	-11.96 ± 0.33	15.09 ± 0.40	8.86 ± 0.15	-157.73 ± 0.04
CP-Flow	-0.52 ± 0.01	-10.36 ± 0.03	16.93 ± 0.08	10.58 ± 0.07	-154.99 ± 0.08

Table 6: Test negative log-likelihood (in nats) of tabular datasets in Papamakarios et al. (2017) for density estimation models (lower is better). Results for compared models are taken from Grathwohl et al. (2018). Average and standard deviation report over 3 random seeds at the best hyperparameters found by grid search. For CP-Flow trained on GAS and BSDS300, only one seed converged at the time of submission, so we report N/A for the standard deviation.

Dataset	N. FLOWS	N. HIDDEN LAYERS	N. HIDDEN UNITS	N. PARAMETERS
POWER	10	5	512	5,463,272
GAS	5	5	512	2,757,276
HEPMASS	5	5	512	2,923,897
MINIBOONE	2	5	256	379,232
BSDS300	10	5	256	2,152,456

Table 7: Best hyperparameters found in our search and the consequent total number of model parameters.

For each dataset, we search via grid search for the best hyperparameter configuration by calculating the negative log-likelihood in the validation set using an exact brute-force computation of the log-determinant of the Jacobian. Initially, we focus our search at basic hyperparameters that influence the total number of parameters of the model, such as the number of flow blocks, the number of hidden layers of each convex potential block, and the number of hidden units per layer of each block. After a first selection of candidate hyperparameters after a constant number of training steps, we instantiate extra experiments considering variations in the softplus-type activations we use. We find that Gaussian-softplus and symmetrizing it in the activations of the first layer help to the performance, oftentimes accelerating the training. In table 7, we report the final hyperparameter combinations we have used for the results presented in table 6.

E.5 GENERATIVE MODELING WITH CONVOLUTIONAL ICNN

On MNIST and CIFAR-10, we used a multiscale architecture. For MNIST, we had 8 CP-Flow blocks, followed by an invertible downsampling (Dinh et al., 2017), followed by 8 CP-Flow blocks, another downsampling, and final 8 CP-Flow blocks. Before every block was an ActNorm (Kingma & Dhariwal, 2018) layer. For CIFAR-10, we had 2 CP-Flow blocks, followed by an invertible downsampling (Dinh et al., 2017), followed by 2 CP-Flow blocks, downsampling, 2 CP-Flow blocks, downsampling, and final 2 CP-Flow blocks. Before every CP-Flow block was an ActNorm (Kingma & Dhariwal, 2018) layer. All ICNN architectures had 4 hidden layers and 64 hidden units wide. We averaged across the final spatial dimensions to obtain a scalar output for the convex potential.

We test the invertibility of a CP-Flow model trained on CIFAR-10 on a set of out-of-distribution data sets constructed by Behrmann et al. (2020) in Table 8. Notably, we do not suffer from the exploding



Figure 6: (top) Data samples. (bottom) Reconstruction after passing it through a CP-Flow.

DATASET	GLOW	RESFLOW	CP-FLOW
CIFAR-10 (in-dist)	1.12E-6	5.16E-4	1.96E-3
Uniform	Inf	3.04E-4	3.62E-3
Gaussian	Inf	1.31E-4	5.97E-3
Rademacher	Inf	3.43E-5	5.27E-3
SVHN (Netzer et al., 2011)	9.94E-7	1.31E-3	2.53E-3
Texture (Cimpoi et al., 2014)	Inf	3.66E-4	1.69E-3
Places (Zhou et al., 2017)	Inf	5.31E-4	1.76E-3
tinyImageNet	Inf	6.26E-4	1.65E-3

Table 8: Reconstruction RMSE. Results for Glow and ResFlow are taken from Behrmann et al. (2020).

inverse problem and can reliably invert all data sets, either in or out of distribution. Samples of reconstructed images are shown in Figure 6 which show no visual difference between the original images and their reconstructions.

E.6 AMORTIZING ICNN FOR VARIATIONAL INFERNECE

We use the partially input convex neural network from Amos et al. (2017) with multiplicative conditioning. There are some other options for conditioning, such as with the hypernetwork (Ha et al., 2016) or feature-wise transformation (Dumoulin et al., 2018). We remove the non-linear path of the conditioned variable beyond the first layer, since we found that it hurts training of the VAE. We use the Gaussian-softplus for all the experiments, symmetrizing it on the first layers. For the Caltech experiment, we also use the offset version, and the softplus is initialized with a multiplicative constant of 2, which we found leads to faster convergence.

Data	N. FLOWS	N. HIDDEN LAYERS	N. HIDDEN UNITS
FreyFaces	4	4	256
Omniglot	2	2	512
Caltech	8	4	256

Table 9: Architectural details for the VAE experiment.

Model	FREYFACES	OMNIGLOT	CALTECH
No flow (Kingma & Welling, 2013)	4.53 $\pm$ 0.02	104.28 $\pm$ 0.39	110.80 $\pm$ 0.74
Planar (Rezende & Mohamed, 2015)	4.40 $\pm$ 0.06	102.65 $\pm$ 0.42	109.66 $\pm$ 0.42
IAF (Kingma et al., 2016)	4.47 $\pm$ 0.05	102.41 $\pm$ 0.04	111.58 $\pm$ 0.38
Sylvester (Van Den Berg et al., 2018)	4.45 $\pm$ 0.04	99.00 $\pm$ 0.04	104.62 $\pm$ 0.29
CP-Flow (Ours)	4.47 $\pm$ 0.02	102.06 $\pm$ 0.03	106.53 $\pm$ 0.55
CP-Flow aug (Ours)	4.45 $\pm$ 0.03	100.82 $\pm$ 0.30	105.17 $\pm$ 0.57

Table 10: Negative ELBO of VAE (lower is better). For FREYFACES the results are in bits per dim. The numbers are averaged over three runs of experiments. Standard deviation is presented in the appendix E.6. For CP-Flow with input-augmented ICNN trained on OMNIGLOT, only one seed converged at the time of submission, so we report N/A for the standard deviation.

F ADDITIONAL ABLATION

We perform an analysis on the effect of changing the absolute error tolerance level (atol) on the gradient estimator’s bias and variance. In a synthetic setting, we sample a $d \times d$ (where $d = 10$) positive definite matrix H from $\mathcal{W}(I, 2d)$ and scale it down by $1/d$ (so that the diagonal entries do not grow as d increases). We linearly interpolate atol values between 0 and 1, and compare the error statistics of the gradient estimator against the ground truth gradient of $\log \det H$ w.r.t H . The error statistics are (1) the signal-to-noise ratio (expected value of the estimator, or the ground truth gradient, divided by the standard deviation), and (2) the absolute bias (expected value of the absolute difference between the estimate and the ground truth). Since these quantities are intractable, we use Monte Carlo estimate of 100K samples (draw 100K i.i.d. samples of the gradient estimator) to smooth out the curves. The estimates are averaged across all entries of H and 10 different random H ’s. Results are presented in fig. 7.

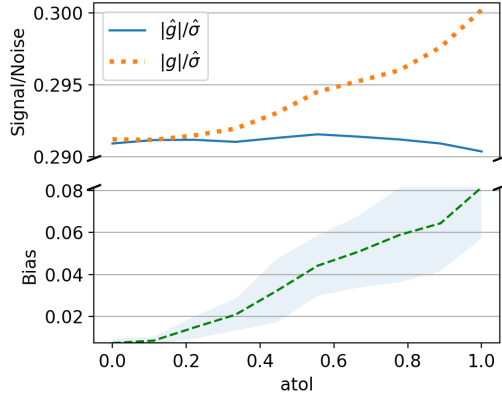


Figure 7: Simulated signal-to-noise ratio and bias of the gradient estimator for different atol values

On a more realistic setting, we monitor the average number of CG iterates (hvp calls), per-iteration time, as well as validation loss on the Miniboone dataset. Meanwhile, we compare the vanilla ICNN, input-augmented ICNN, as well as a dense version of ICNN (Huang et al., 2017). Figure 8 shows that as tolerance value becomes smaller, we indeed need more hvp calls, which eventually saturates at the dimensionality of the data. However, there is not much difference in terms of log-likelihood if the tolerance is sufficiently small: the validation loss oscillates and diverges for $\text{atol} = 0.1$, while the curves for different atol values smaller than 0.001 are almost indistinguishable and fairly stable. On the other hand, there is a noticeable difference in performance if we simply replace the vanilla ICNN with the input-augmented ICNN or the dense ICNN. This suggests *tuning the architecture of the convex potential is more crucial for improving the overall performance*. This set of ablation studies were all performed on P100 NVIDIA GPUs and with a constant batch size of 1024.

Runtime of directly backpropagating through Lanczos Furthermore, we include an analysis on the effect of the number of eigenvalues used in the stochastic Lanczos quadrature method (*i.e.* size of the tridiagonal matrix) on training time. In this experiment we directly backpropagated through the Lanczos estimate to compute the stochastic gradient (instead of using the proposed gradient estimator). Figure 9 shows that the runtime is much higher than the runtime of the proposed gradient estimator using CG (bottom left of Figure 8). We note that the experiments with $m = 5$ diverged, possibly due to the error in estimation. This complements the memory profile shown in Figure 2, and accentuates the lower runtime and memory requirement of the proposed method by contrast.

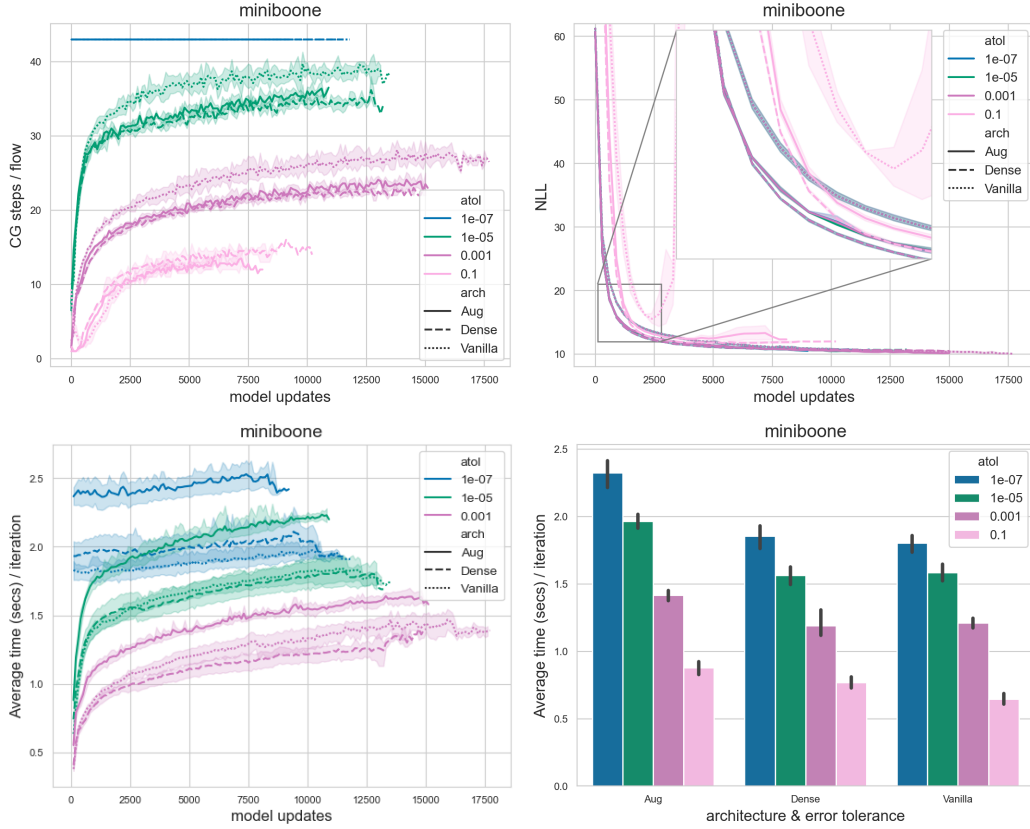


Figure 8: Ablation of different ICNN architectures and absolute error tolerance for conjugate gradient. Left: the average number of CG iterates (hvp calls) per flow layer (top row) and the corresponding average time (in seconds) per iteration (bottom row). Top right: validation set negative log-likelihood (exact estimate). Notice that, for $\text{atol} = 1e-7$, CG iterations cap at 43 per flow layer; this is the dimensionality of the input data in the MINIBONE dataset. Bottom right: per-iteration time (in seconds) averaged over all training steps.

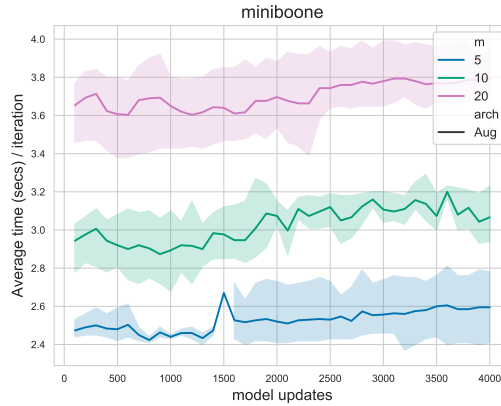


Figure 9: Ablation of the effect of number of eigenvalues used in SLQ on average time (in seconds) per iteration.